

Универзитет „Св. Кирил и Методиј“ во Скопје

**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И КОМПЈУТЕРСКО
ИНЖЕНЕРСТВО**

Континуирана интеграција и испорака

EduEval

*Student Grading App with LLMs — контејнеризација, CI/CD и оркестрација со
Kubernetes*

Студент: Христијан Мијалков

Факултет: ФИНКИ — Скопје

20.08.2026

Преглед

EduEval (во интерфејсот именуван „EduBot“) е full-stack веб-апликација која на студентите им дава формативна повратна информација за нивните вежби, генерирана со голем јазичен модел (LLM). Студентот испраќа текстуален промпт и/или слика од својата работа преку multipart/form-data барање; backend-от го проследува до LLM провајдер (OpenRouter, модел google/gemini-2.0-flash-exp:free, OpenAI-компатибилен формат) кој одговара со формативна повратна информација, но е системски инсториран да одбие помош при активни тестови/испити.

Апликацијата е составена од посебни пакети: frontend (Angular 17, Tailwind CSS, служен преку Nginx), backend (Spring Boot 3.4.4 / Java, Spring Security + JWT автентикација) и база на податоци (MySQL 8.0). Оваа документација се фокусира на докеризацијата, Kubernetes deployment архитектурата и CI/CD пайплајнот.

Архитектура

Компоненти

- Frontend: статичка Angular апликација, сервирана преку Nginx контејнер (истиот Nginx го проксира /api сообраќајот кон backend-от — само во Docker Compose контекст)
- Backend: Spring Boot REST API на порт 8080, со Spring Security + JWT филтер на секое барање надвор од /api/auth/**
- Database: MySQL 8.0 StatefulSet со персистентно складирање (PVC)
- Ingress: Nginx Ingress Controller за рутирање на надворешниот сообраќај

Kubernetes Namespace

Сите ресурси се деплојираат во посебен namespace, наречен edueval, за подобра изолација и организација, одвоено од другите системски namespace-и (kube-system, ingress-nginx и сл.).

```
apiVersion: v1
kind: Namespace
metadata:
  name: edueval
  labels:
    app.kubernetes.io/part-of: student-grading-app
```

Слика 1: namespace.yaml

Докеризација

Секој сервис е пакуван во посебен Docker image. Двата Dockerfile-и се наоѓаат во коренот на репозиториумот и се градат со коренот како build context.

Backend Dockerfile

Еднофазна изградба: base image eclipse-temurin:21-jdk-jammy, ./mvnw package го гради runnable jar-от, апликацијата стартува директно со java -jar.

```
FROM eclipse-temurin:21-jdk-jammy

WORKDIR /app
COPY backend/ .

RUN ./mvnw package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "target/backend-0.0.1-SNAPSHOT.jar"]
```

Слика 2: Dockerfile-backend

Frontend Dockerfile

Двофазна (multi-stage) изградба: првиот стадиум (node:20) ги инсталира зависностите со npm install и прави production build на апликацијата; вториот стадиум (nginx:alpine) само ги сервира готовите статички фајлови.

```
FROM node:20 AS builder

WORKDIR /app
COPY frontend/package*.json ./
RUN npm install
COPY frontend/ .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist/frontend/browser /usr/share/nginx/html
COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Слика 3: Dockerfile-frontend

Nginx конфигурација

- location / користи try_files со fallback на /index.html, за да работи client-side рутирањето на Angular SPA
- location /api/ го проксира сообраќајот кон http://backend:8080/api/ (важи само во Docker Compose; во Kubernetes, Ingress-от интервенира пред да стигне до овој Nginx)

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location /api/ {
        proxy_pass http://backend:8080/api;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 120s;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Слика 4: frontend/nginx.conf

```
$ docker images | grep howlinman
howlinman/frontend   latest      83e009f5b1e3   62MB
howlinman/backend    latest     141adda49e45   611MB
```

Слика 5: Изградени Docker images (docker images)

Kubernetes Манифести

Манифестите се наоѓаат во директориумот `k8s/` и се применети по редослед: `namespace` → `configmap/secret` → `mysql (statefulset + service)` → `backend (deployment + service)` → `frontend (deployment + service)` → `ingress`.

1. ConfigMap и Secret

Несензитивните конфигурации (име на база, ниво на логирање, URL на LLM провајдерот) се чуваат во ConfigMap, а лозинките и клучевите во Kubernetes Secret, одвоени од сликите и од Deployment манифестите. Вистинските вредности во Secret никогаш не се commit-нати во git — во pipeline-от се внесуваат од GitHub Actions Secrets при deploy.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: edueval
data:
  DB_NAME: student_db
  DDL_AUTO: update
  OPENAI_URL: https://openrouter.ai/api/v1/chat/completions
  LOG_LEVEL_SPRING_SECURITY: INFO
  LOG_LEVEL_APP: INFO

apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: edueval
type: Opaque
stringData:
  DB_USERNAME: root
  DB_PASSWORD: <inserted-by-CI-from-GitHub-Secrets>
  JWT_SECRET: <inserted-by-CI-from-GitHub-Secrets>
  OPENAI_API_KEY: <inserted-by-CI-from-GitHub-Secrets>

$ kubectl get configmap,secret -n edueval
NAME                                DATA  AGE
configmap/app-config                5      24h
configmap/kube-root-ca.crt          1      24h

NAME                                TYPE    DATA  AGE
secret/app-secrets                  Opaque  4      24h
```

Слика 6: ConfigMap и Secret во namespace edueval

2. MySQL StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
  namespace: edueval
spec:
  serviceName: mysql
  replicas: 1
  selector: {matchLabels: {app: mysql}}
  template:
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          envFrom-еквивалент преку secretKeyRef/configMapKeyRef:
            MYSQL_ROOT_PASSWORD ← Secret.DB_PASSWORD
            MYSQL_DATABASE ← ConfigMap.DB_NAME
          volumeMounts:
            - {name: mysql-data, mountPath: /var/lib/mysql}
          readinessProbe: {tcpSocket: {port: 3306}, initialDelaySeconds: 30}
          livenessProbe: {tcpSocket: {port: 3306}, initialDelaySeconds: 60}
      volumeClaimTemplates:
        - metadata: {name: mysql-data}
          spec: {accessModes: [ReadWriteOnce], resources: {requests: {storage: 2Gi}}}
```

Слика 7: mysql-statefulset.yaml (скратено; целосниот фајл е во репозиториумот)

Клучни карактеристики:

- Користи StatefulSet, бидејќи базата бара стабилен идентитет и сопствено персистентно складирање кое преживува рестарт и повторно распоредување на подот
- 2Gi PersistentVolumeClaim по под, автоматски создаден преку volumeClaimTemplates
- Конфигурација од ConfigMap и Secret преку secretKeyRef/configMapKeyRef

- Headless Service (clusterIP: None) за стабилна DNS идентификација: mysql-0.mysql.edueval.svc.cluster.local — токму на ова име backend-от се поврзува

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: edueval
spec:
  clusterIP: None
  selector: {app: mysql}
  ports:
    - {port: 3306, targetPort: 3306}

$ kubectl get statefulset,pvc -n edueval
NAME                                READY   AGE
statefulset.apps/mysql             1/1     23h

NAME                                STATUS    CAPACITY   ACCESS MODES
persistentvolumeclaim/mysql-data-mysql-0  Bound    2Gi        RWO
```

Слика 8: MySQL StatefulSet и PersistentVolumeClaim

3. Backend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: backend, namespace: edueval}
spec:
  replicas: 1
  selector: {matchLabels: {app: backend}}
  template:
    spec:
      containers:
        - name: backend
          image: howlinman/backend:latest
          imagePullPolicy: IfNotPresent
          ports: [{containerPort: 8080}]
          env:
            - {name: SPRING_DATASOURCE_URL,
              value: jdbc:mysql://mysql-0.mysql.edueval.svc.cluster.local:3306/student_db}
            - {name: SPRING_DATASOURCE_USERNAME, valueFrom: {secretKeyRef: {name: app-secrets, key: DB_USERNAME}}}
            - {name: SPRING_DATASOURCE_PASSWORD, valueFrom: {secretKeyRef: {name: app-secrets, key: DB_PASSWORD}}}
            - {name: JWT_SECRET, valueFrom: {secretKeyRef: {name: app-secrets, key: JWT_SECRET}}}
            - {name: OPENAI_API_KEY, valueFrom: {secretKeyRef: {name: app-secrets, key: OPENAI_API_KEY}}}
            # ... останатите (DDL_AUTO, лог нивоа, OPENAI_API_URL) следат иста шема од ConfigMap
          readinessProbe: {tcpSocket: {port: 8080}, initialDelaySeconds: 60}
          livenessProbe: {tcpSocket: {port: 8080}, initialDelaySeconds: 90}
```

Слика 9: backend-deployment.yaml (скратено; целиот файл е во репозиториумот)

Клучни карактеристики:

- Environment променливи од ConfigMap и Secret преку secretKeyRef/configMapKeyRef — ниту едно чувствително поле не е хардкодирано во манифестот
- TCP readiness/liveness probes на порт 8080
- imagePullPolicy: IfNotPresent — точно, бидејќи deploy задачата закотвува конкретен commit-SHA таг пред apply, па неменливоста на тагот го прави тоа коректно
- Image објавен на DockerHub (howlinman/backend), објавен автоматски од CI/CD пајплајнот
- ClusterIP Service на порт 8080 — backend-от е достапен само внатре во кластерот, преку Ingress

```
apiVersion: v1
kind: Service
metadata: {name: backend, namespace: edueval}
spec:
  selector: {app: backend}
  ports:
    - {name: http, port: 8080, targetPort: 8080}
```

4. Frontend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: frontend, namespace: edueval}
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: frontend
          image: howlinman/frontend:latest
          imagePullPolicy: IfNotPresent
          ports: [{containerPort: 80}]
          readinessProbe: {httpGet: {path: /, port: 80}, initialDelaySeconds: 10}
          livenessProbe: {httpGet: {path: /, port: 80}, initialDelaySeconds: 20}
---
apiVersion: v1
kind: Service
metadata: {name: frontend, namespace: edueval}
spec:
  selector: {app: frontend}
  ports:
    - {name: http, port: 80, targetPort: 80}
```

Слика 10: frontend-deployment.yaml + frontend-service.yaml

Клучни карактеристики:

- Nginx serving на порт 80; frontend-от е статичка апликација и не бара сопствени ConfigMap/Secret
- Комуницира со backend-от преку релативни /api/... барања (utils/consts.ts), па не му е потребен hostname на backend-от — рутирањето е одговорност на Ingress-от
- HTTP readiness/liveness probes на „/“

5. Services

Service	Тип	Порт	Намена
mysql	Headless ClusterIP	3306	Стабилна DNS идентификација за StatefulSet-от
backend	ClusterIP	8080	REST API, достапен преку Ingress
frontend	ClusterIP	80	Nginx со статичките фајлови на Angular

6. Ingress

Ingress-от е единствената влезна точка во апликацијата. Рутирањето е базирано на патека: сè под /api оди директно кон backend Service-от, а сè останато кон frontend-от. Не е потребно препишување на патеки (path rewriting), бидејќи backend-от самиот очекува барања на /api/**.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: edueval-ingress
  namespace: edueval
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m" # за прикачување слики
spec:
  ingressClassName: nginx
  rules:
    - host: edueval.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend: {service: {name: backend, port: {number: 8080}}}
          - path: /
            pathType: Prefix
            backend: {service: {name: frontend, port: {number: 80}}}
```

Слика 11: ingress.yaml

Рутирање:

- /api/* → backend Service (порт 8080)
- /* → frontend Service (порт 80)
- Host: edueval.local (се додава во /etc/hosts и покажува кон IP адресата на Minikube)

```
$ kubectl describe ingress edueval-ingress -n edueval
Name:          edueval-ingress
Namespace:     edueval
Address:       192.168.49.2
Ingress Class: nginx
Rules:
  Host          Path  Backends
  ----          -
  edueval.local /api  backend:8080 (10.244.0.88:8080)
               /    frontend:80 (10.244.0.89:80)
```

Слика 12: Ingress конфигурација и рутирање

CI/CD Pipeline (GitHub Actions)

Workflow структура

Пајплајнот е дефиниран во еден фајл, `.github/workflows/ci-cd.yml`, и се активира на push и pull request кон main гранката. Се состои од две задачи:

- build-and-push — изградба на двете Docker images (backend и frontend) за две архитектури (linux/amd64 и linux/arm64, преку QEMU + Buildx) и push на DockerHub, означени со :latest и <git-sha>. Работи на GitHub-хостиран ubuntu-latest runner.
- deploy — применува ги Kubernetes манифестите, ги ажурира Secrets идемпотентно (`--dry-run=client | kubectl apply`), го закотвува :latest тагот на точниот commit SHA, и чека (`kubectl rollout status`) новите подови да станат подготвени. Работи на self-hosted runner, регистриран на истиот компјутер каде работи Minikube — потребно бидејќи GitHub-облак runner-ите немаат мрежен пат до кластер на localhost.

Конфигурација на job-от

```
name: CI/CD Pipeline
on:
  push: {branches: [main]}
  pull_request: {branches: [main]}

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
      with: {username: ${ secrets.DOCKERHUB_USERNAME }, password: ${ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v6 # x2, backend + frontend
      with: {platforms: linux/amd64,linux/arm64, push: ${ github.ref == 'refs/heads/main' }}

  deploy:
    needs: build-and-push
    runs-on: self-hosted
    if: github.ref == 'refs/heads/main'
    steps:
      - run: kubectl cluster-info
      - run: kubectl apply -f k8s/namespace.yaml
      - run: |
          kubectl create secret generic app-secrets \
            --from-literal=DB_PASSWORD=${ secrets.DB_PASSWORD } ... \
            --dry-run=client -o yaml | kubectl apply -f -
      - run: sed -i ' ' "s|:latest|:${ github.sha }|g" k8s/*-deployment.yaml
      - run: kubectl apply -f k8s/*.yaml
      - run: kubectl rollout status deployment/backend --timeout=420s
```

Слика 13: `.github/workflows/ci-cd.yml` (скратено; целосниот фајл е во репозиториумот)

Функционалности

- Изградба на Docker images за две архитектури (amd64 + arm64) со Docker Buildx + QEMU
- Push на DockerHub со :latest и commit-SHA тагови (за rollback и следење на верзии)
- Build cache преку GitHub Actions cache (cache-from / cache-to: type=gha), за побрзи следни градби
- Условна најава и push: само за push настани кон main, не и за pull request-и
- Целосен CD дел (бонус): автоматско применување на манифестите и чекање rollout да заврши, без рачна интервенција по push

GitHub Secrets

Пајплајнот бара рачно креирани secrets во поставките на репозиториумот (Settings → Secrets and variables → Actions):

Secret	Опис
DOCKERHUB_USERNAME	Docker Hub корисничко име, дел од името на image-от
DOCKERHUB_TOKEN	Docker Hub access token (за автентикација при push)
DB_USERNAME / DB_PASSWORD	Креденцијали за MySQL, инјектирани во Kubernetes Secret при deploy
JWT_SECRET	Клуч за потпишување на JWT токени

OPENAI_API_KEY	OpenRouter API ключ за LLM повиците
----------------	-------------------------------------

```
$ gh secret list --repo HowlinMan24/Student-Grading-App-with-LLMs
DB_PASSWORD          2026-05-23
DB_USERNAME          2026-05-23
DOCKERHUB_TOKEN      2026-09-18
DOCKERHUB_USERNAME   2026-05-23
JWT_SECRET           2026-05-23
OPENAI_API_KEY       2026-09-18
```

Слика 14: Регистрирани GitHub Actions Secrets

Демонстрација

Апликацијата е деплојирана на локален Kubernetes кластер (Minikube), при што сите подови се во статус Running во namespace-от edueval:

```
$ kubectl get pods -n edueval
NAME                                READY   STATUS    RESTARTS   AGE
backend-597d9ccbd6-kqdbz           1/1     Running   0           11h
frontend-6b4b8d9475-44wb7         1/1     Running   0           11h
mysql-0                             1/1     Running   0           23h
```

Слика 15: Подови во namespace edueval

Проверката на HTTP-рутирањето преку самиот Ingress (со Host заглавие, без порт-проследување) потврди дека „/“ ја служи фронтендот, а „/api/“ директно го достигнува backend-от: барање до заштитена рута без JWT токен коректно врати HTTP 401 (Unauthorized) — доказ дека Security-филтерот работи и во кластерот, не само локално.

Клучни проблеми решени при изработката

Целосниот дневник на проблеми (со точни логови и commit-и) е достапен во `DEVOPS_DOCUMENTATION.md` во репозиториумот. Накратко:

- Недостасувачки `nginx.conf` — Docker build постојано паѓаше бидејќи фајлот никогаш не беше commit-нат. Решено со commit.
- Хардкодирани тајни во јавен репозиториум — `application.properties` привремено содржеше вистински лозинки. Вратено на читање преку `environment` променливи пред commit.
- CD задачата не можеше да го достигне Minikube — GitHub-облак runner нема мрежен пат до кластер на localhost. Решено со регистрирање self-hosted runner.
- Несовпаѓање на архитектурата (arm64 vs amd64) — images градени само за amd64 не можеа да се повлечат на Apple Silicon Mac. Решено со мулти-архитектурна изградба.
- Застарена лозинка во MySQL volume — контејнерот на базата беше стар 3 месеци и никогаш повторно иницијализиран, па задржа стара root-лозинка. Решено со `docker compose down -v` и повторна иницијализација.

Линкови

GitHub репозиториум:

github.com/HowlinMan24/Student-Grading-App-with-LLMs

Backend Docker image (Docker Hub):

hub.docker.com/r/howlinman/backend

Frontend Docker image (Docker Hub):

hub.docker.com/r/howlinman/frontend

GitHub Actions Pipeline:

github.com/HowlinMan24/Student-Grading-App-with-LLMs/actions

Заклучок

Проектот EduEval целосно ги исполнува бараните точки: апликацијата е поставена на јавен git репозиториум, докеризирана, оркестрирана со Docker Compose, автоматски градена и објавувана преку GitHub Actions CI/CD pipeline (со дополнителен CD дел за бонус поени), и распоредена во посебен namespace во Kubernetes со Deployment, Service, Ingress и StatefulSet, сите со сопствени ConfigMap/Secret. Работата беше проверена со реално извршување и со верификација дека системот работи од крај до крај.